

Deduplicating Source Locations Across Loaded Modules

Cutting loaded SourceLocation pressure in Clang

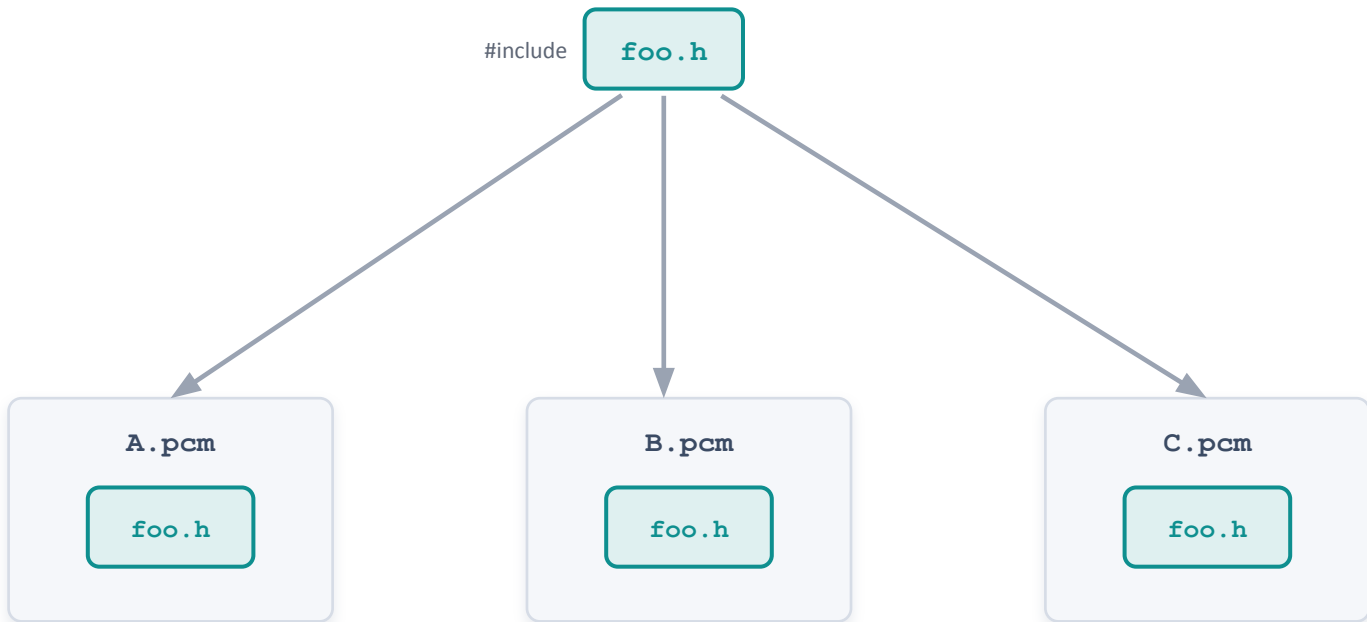


Ayokunle Amodu

The C++ Alliance / Compiler Research Group
Mentor: Vassil Vassilev

MIDTERM CHECKPOINT

One header, many modules

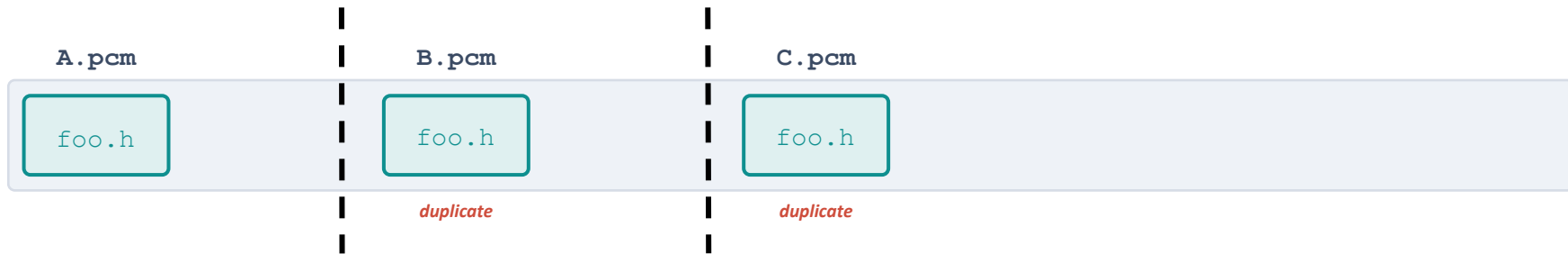


A header included by many modules is serialized into each of their PCM.

THE PROBLEM

Every copy takes real address space

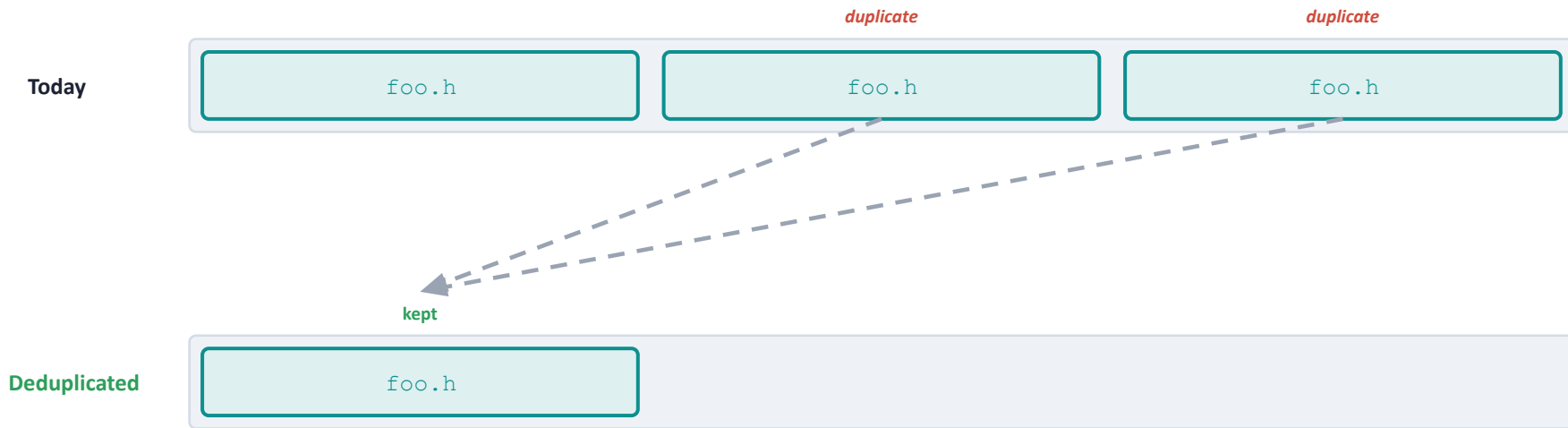
loaded SourceLocation address space



Two of three ranges are pure waste.

On a large -fmodules build this duplication is one of the things that pushes you toward **source-location exhaustion**.

Before → After



When a file is already loaded, later modules reuse that copy instead of reserving a fresh range.

When are two entries the same file?

First plan



`FileEntryRef`

Resolving the file entry means unnecessary open syscalls at load.



What we do



`name + size`

Read straight from the serialized entry metadata.

size 812

`foo.h`

=

size 812

`foo.h`

same name & size → same file

size 812

`a.h`

≠

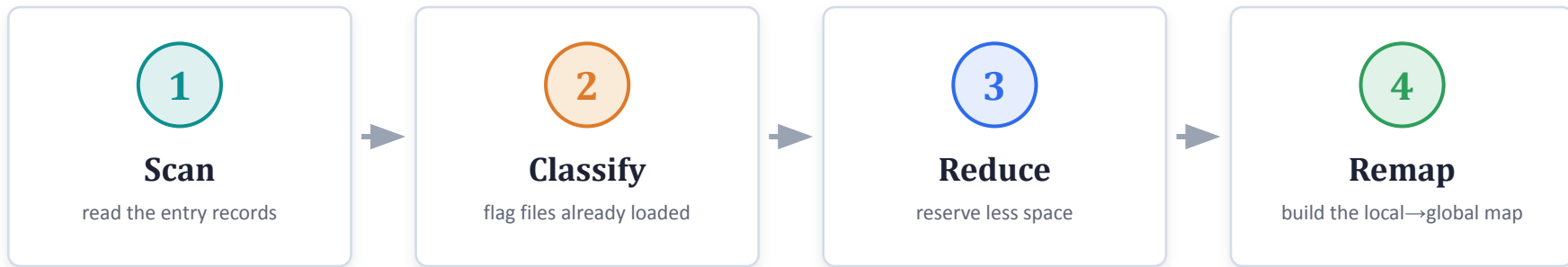
size 912

`a.h`

different name → stays distinct

size guards against two modules built against different versions of the same file

What happens when a module loads



Scan the entries, flag the duplicates

B's SLoc file entries	identity check
b.h <i>new</i>	<i>keep</i>
foo.h <i>already loaded (from A)</i>	<i>duplicate</i>
main.cpp <i>new</i>	<i>keep</i>

running tally

1

duplicate entry

50 bytes

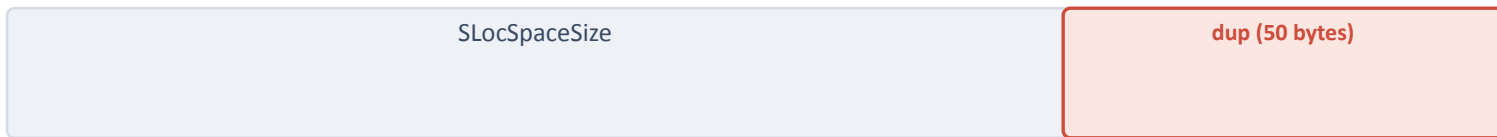
would-be duplicate space

Each entry is given its name + size identity and checked against files already loaded. Matches are flagged, and the space they would have taken is tallied.

STEP 3

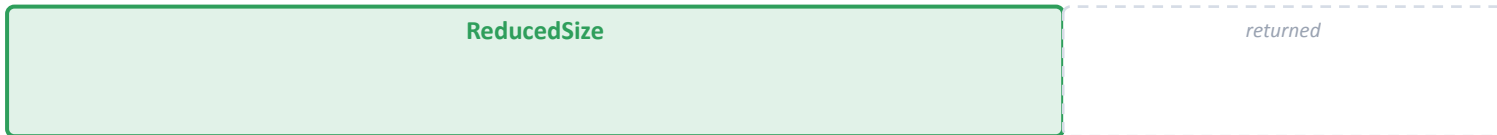
Reserve a smaller range up front

would reserve



subtract duplicate bytes

actually reserve



`ReducedSize = SLocSpaceSize - duplicate bytes`
Because the duplicates are already known, the module claims a smaller slice.

The old map: one flat shift



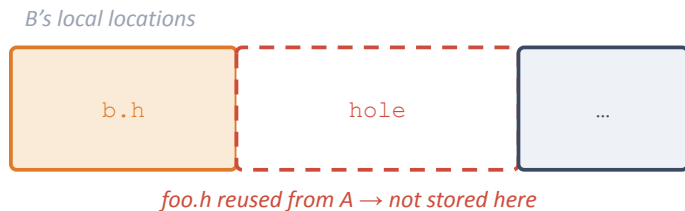
`global = local + baseOffset`. A module's entries were one contiguous block, so a single add slid the whole block into loaded space.

A location, the old way



`local 65` → `65 + 200 = 265`.
The location lands in B's own private copy of foo.h.

Now: a reused file leaves a hole

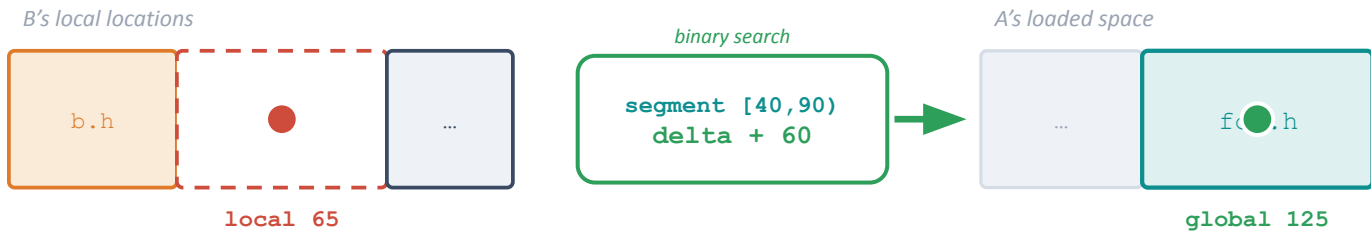


B's segment table (sorted by local start)

local range	maps to
[0, 40)	b.h → B's block +200
[40, 90)	foo.h → A's copy +60
[90, 120)	... → B's block +150

Each module carries a short sorted table of segments. Kept files shift into their own block; the reused file redirects into A. Translating a location is a **binary search** for the covering segment.

The same location, now



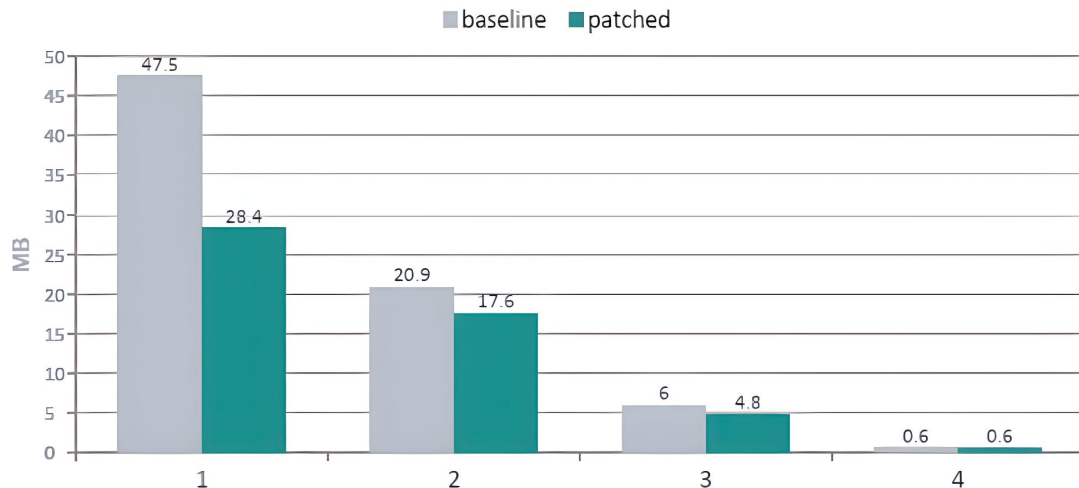
Old `local 65` → `265` *its own copy*

New `local 65` → `125` *A's shared copy*

With nothing deduplicated the table is a single segment = the old flat shift.

RESULTS

Loaded SLoc space, measured on a custom Boost bench



Self-contained libraries with nothing shared show 0% i.e., the dedup does nothing when there is nothing to reuse.

40%

less loaded SLoc space on the combined main TU

asio + beast + spirit + hana · 47.5 → 28.4 MB

Compile time **flat (−0.9%)**

Peak RSS **flat (+1.2%)**

stat / open calls **identical**

config (nothing shared) **0%**

40% is not a magic number

a file shared by n modules

$$\frac{n - 1}{n}$$

50%

2 modules

90%

10 modules

→ 100%

many

The 40% just reflects how much this particular module set happens to share.

More shared headers → a bigger number on the same code.

Where it stands

1

Posted & under review

It's up on upstream; maintainers testing it against their internal codebases.

2

Reduce peak RSS

Right now RSS ticks up slightly on our custom bench; find out why and bring it down.

3

Reused-entry include chain

Confirm a redirected file inheriting the primary's "included from" info is fine as-is, or add a fix if not.